

RAG Week Assignment

Section A: Vector Databases • Section C: Advanced RAG

Name: Rohit Bagal

Date: 20 September 2026

Section A: Vector Databases

A1. Explain in your own words what a vector database is and how it is different from a traditional database like MySQL or PostgreSQL.

A vector database stores *embeddings*, which are numerical vectors that represent the meaning of a piece of text. Here we do not look for an exact match; we look for the **nearest neighbours** of the query vector. In MySQL, data is stored in a structured way, in rows and columns.

	MySQL / PostgreSQL	Vector database
Storage	Rows and columns (structured data)	Embeddings representing meaning
Query type	Exact match, range and joins	Nearest-neighbour search
Result	Rows that satisfy the condition	Top- <i>K</i> most similar items
Used in	Web applications, transactional and payment systems	RAG, recommendation systems, AI applications

A2. Take any sentence of your choice. Describe what happens to it step by step from raw text to being stored in a vector database. Name each stage.

Example sentence: “Machine learning helps computers learn from data.”

1. **Raw text.** The original sentence is taken as input: “Machine learning helps computers learn from data.”
2. **Text preprocessing.** The text is cleaned by removing unnecessary spaces, special characters or unwanted content.
3. **Chunking.** A long document is divided into smaller meaningful pieces called chunks. A single short sentence like this one can stay as one chunk.
4. **Embedding generation.** An embedding model converts the text into a numerical vector that represents its meaning, e.g. [0.21, -0.45, 0.78, ...].

5. **Metadata creation.** Extra information is stored with the vector, such as document ID, page number, source or timestamp.
6. **Vector storage.** The vector, the original chunk and its metadata are stored in a vector database such as Pinecone, Milvus, Chroma or Qdrant.
7. **Similarity search.** Later, when a user asks a question, the query is also converted into an embedding. The database compares it with the stored vectors using measures such as cosine similarity and retrieves the most relevant chunks.

Reference: <https://www.pinecone.io/learn/retrieval-augmented-generation/>

A3. Why can't we just use a normal database index (like a B-tree) to search through embeddings? Explain the core difference between exact lookup and similarity search.

A B-tree is used when we want to find an **exact value** or a value within a **particular range**. With embeddings we are not looking for an exact vector; we want the vectors that are **most similar or closest** to our query. For example, a B-tree can find ID = 101, but a vector search can find a document whose meaning is similar to our question.

	Exact lookup (B-tree)	Similarity search
Finds	Exact or ordered values	The closest / most similar vectors
Condition	Equality or range conditions	Distance measures such as cosine similarity or Euclidean distance

So a normal B-tree is not suitable for efficiently searching embeddings. Vector indexes such as **HNSW** are commonly used for this purpose.

A4. You are given two vectors $A = [1, 2, 3]$ and $B = [2, 4, 6]$. Calculate the cosine similarity between A and B by hand, showing your steps. What does the result tell you about these two vectors?

Formula:

$$\cos(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

Step 1: Dot product

$$\begin{aligned} A \cdot B &= (1 \times 2) + (2 \times 4) \\ &\quad + (3 \times 6) \\ &= 2 + 8 + 18 = 28 \end{aligned}$$

Step 2: Magnitudes

$$\begin{aligned} \|A\| &= \sqrt{1^2 + 2^2 + 3^2} \\ &= \sqrt{14} \\ \|B\| &= \sqrt{2^2 + 4^2 + 6^2} \\ &= \sqrt{56} = 2\sqrt{14} \end{aligned}$$

Step 3: Cosine similarity

$$\begin{aligned} \cos(A, B) &= \frac{28}{\sqrt{14} \times 2\sqrt{14}} \\ &= \frac{28}{28} = 1 \end{aligned}$$

AS. $A = [1, 2, 3]$, $B = [2, 4, 6]$

Formula for co-sine similarity:-

$$= \frac{(A \cdot B)}{\|A\| \times \|B\|} \leftarrow \text{dot product}$$

Step 1:- Calculate dot product

$$\begin{aligned} A \cdot B &= (1 \times 2) + (2 \times 4) + (3 \times 6) \\ &= 2 + 8 + 18 \\ &= 28 \end{aligned}$$

Step 2:- Calculate the magnitude of A & B

$$\begin{aligned} \|A\| &= \sqrt{1^2 + 2^2 + 3^2} = \sqrt{14} & \|B\| &= \sqrt{2^2 + 4^2 + 6^2} \\ & & &= \sqrt{56} = 2\sqrt{14} \end{aligned}$$

Step 3:- Calculate co-sine similarity

$$\cos = \frac{28}{\sqrt{14} \times 2\sqrt{14}} = \frac{28}{28} = 1$$

Cosine Similarity = 1

This means the two vectors have the exactly same direction. B is simply 2 times A , so they are perfectly similar in terms of directions.

My handwritten working

Result: cosine similarity = 1.

This means the two vectors have exactly the same direction. B is simply 2 times A , so they are perfectly similar in terms of direction.

A5. Explain why a vector database is essential for RAG specifically. What would break if you removed it from the pipeline?

A vector database is important in RAG because it stores the embeddings of the documents and helps retrieve the information that is most relevant to the user's question.

How it works in RAG:

1. Documents are divided into chunks.
2. Each chunk is converted into an embedding (vector).
3. These vectors are stored in a vector database.
4. When the user asks a question, the question is also converted into a vector.
5. The vector database finds the most similar chunks.
6. These chunks are given to the LLM as context to generate the answer.

What happens if we remove it? Without a vector database, the RAG system has no efficient way to search and retrieve relevant information from a large collection of documents. The LLM may receive no useful context and would have to rely mainly on its existing knowledge, which can lead to less accurate or unsupported answers.

Section C: Advanced RAG

C1. You are building a RAG system for a company's product manuals. The manuals have text instructions along with diagrams and photos showing how to assemble the product. A user asks "how do I attach the side panel," and the answer is shown entirely in a diagram, with no accompanying text. Explain how you would approach this with multimodal RAG. What would go wrong if you used a text-only RAG pipeline here?

For this case I would use **multimodal RAG**, because the important information is present in an image rather than in text.

Approach:

1. Extract both text and images from the product manual.
2. Process the diagrams and images using a vision model or a multimodal embedding model.
3. Store the text and image embeddings in a suitable vector database, along with metadata such as page number and image location.
4. When the user asks "How do I attach the side panel?", convert the question into an embedding.

5. Search the database for the most relevant text and visual content.
6. Retrieve the diagram showing the side-panel assembly.
7. Give the retrieved diagram and the relevant text to a multimodal LLM, which can understand the image and explain the assembly steps to the user.

What would go wrong with text-only RAG? A text-only pipeline mainly processes text, so the diagram would be ignored. If the instructions exist only in the image, the relevant information will never be retrieved. The system might:

- fail to find the correct assembly instructions;
- give an incomplete answer;
- generate an answer that is not based on the actual diagram.

Main point: when important information exists inside images or diagrams, the RAG system must be able to retrieve and understand visual information, not just text.

C2. Briefly compare the two approaches to multimodal retrieval covered in lecture: unified embedding space vs. separate indexes with fusion. Which one would you pick for the product manual scenario above, and why?

Unified embedding space	Separate indexes + fusion
Text and images are converted into a common vector space.	Text and images are stored in separate indexes.
One search can retrieve both text and images.	Text and images are searched separately.
Usually simpler during retrieval.	Requires an extra step to combine (fuse) the results.
May require a multimodal embedding model, which can be more computationally expensive.	Can use separate lightweight models and may reduce computational cost.

My choice: separate indexes with fusion.

For the product manual scenario I would choose separate indexes with fusion, especially if computational cost is the main concern. Text and images can be processed independently, and their results can be combined only when needed. This gives more flexibility and can be more cost-efficient, depending on the models and the amount of data.